

Instituto Tecnológico de San Luis Potosí

Ingeniería en Sistemas Computacionales

Lenguajes de Interfaz 11:00-12:00

Programación para dispositivos de Salida

Alumna:

Venegas Ordaz Fátima Vianney

Lugar y Fecha:

Soledad de Graciano Sánchez, San Luis Potosí, 11 de abril de 2026

Índice

Introducción.	3
Programación de dispositivos de salida (Impresoras y Cámaras de video)	4
Impresoras.....	4
Cámaras de video.....	6
Conclusión.	8
Referencias.....	9

Introducción.

El presente trabajo tiene como objetivo investigar y analizar el funcionamiento técnico de los dispositivos de salida, los cuales permiten que la computadora comunique los resultados del procesamiento al usuario. A través de esta investigación, se explora cómo el software utiliza lenguajes de programación y APIs específicas para transformar datos digitales en señales físicas, tales como imágenes en un monitor, impresiones en papel o vibraciones sonoras en un altavoz.

Programación de dispositivos de salida (Impresoras y Cámaras de video)

Impresoras.

Una impresora es un dispositivo de salida gestionado por un gestor de impresión de servidor y diseñado para imprimir documentos en papel.

La impresora es un periférico de salida física. Su programación requiere interactuar con el "Gestor de Impresión" del sistema operativo para enviar datos que el hardware pueda convertir en puntos de tinta o tóner. (IBM, s.f.)

Lógica de programación: El software no habla directamente con los motores de la impresora, sino que utiliza una API (como javax.print) para crear un Trabajo de Impresión (Print Job). Este trabajo envía una señal electrónica que la placa de control de la impresora traduce en coordenadas para el cabezal de impresión. (Oracle, 2024)

Código de implementación.

```
import javax.print.*; // Librería para servicios de impresión

import java.io.*;

public class Milmpresora {

    public static void main(String[] args) {

        // Primero, le pedimos a Java que busque la impresora que tenemos de
        fábrica

        PrintService milmpresora =
        PrintServiceLookup.lookupDefaultPrintService();

        if (milmpresora != null) {

            System.out.println("Encontré la impresora: " +
            milmpresora.getName());
```

```

// Aquí definimos que lo que vamos a mandar es texto sencillo
DocFlavor tipoTexto = DocFlavor.INPUT_STREAM.AUTODetect;

// Creamos el "trabajo", que es como la orden para que la impresora
empiece
DocPrintJob tarea = milImpresora.createPrintJob();

// Escribimos lo que queremos que salga en el papel
String texto = "Hola Profe, esta es una prueba de salida desde Java.";

InputStream          flujoDatos          =          new
ByteArrayInputStream(texto.getBytes());

Doc miDoc = new SimpleDoc(flujoDatos, tipoTexto, null);

try {
    // Aquí es donde el programa manda la señal electrónica a la
    impresora

    tarea.print(miDoc, null);

    System.out.println("¡Listo! El texto se envió a la impresora.");
} catch (PrintException e) {
    System.out.println("Hubo un error al conectar con el hardware.");
}
}
}
}

```

Cámaras de video.

Una videocámara o cámara de video es un dispositivo, generalmente portátil, que permite registrar imágenes y sonidos. Convirtiéndolos en señales eléctricas que pueden ser reproducidos por un aparato determinado. En otras palabras, una cámara de vídeo es un traductor óptico.

Su programación consiste en capturar la señal eléctrica que genera la luz al golpear el sensor y mostrarla como una salida digital en tiempo real. (Euroinnova, 2024)

Lógica de programación: El programa utiliza la librería OpenCV para gestionar el hardware. La lente captura fotones que el sensor CMOS convierte en señales eléctricas. El software "lee" estas señales miles de veces por segundo y las traduce en píxeles que forman el video en la pantalla. (OpenCV, 2024)

Código de implementación.

```
import cv2 # Usamos esta librería que es la mejor para manejar cámaras

def usar_camara():

    # Conectamos con la cámara (el 0 significa que es la primera que
    encuentre)

    # Esto le dice al hardware que se encienda

    mi_video = cv2.VideoCapture(0)

    print("Prendiendo la cámara... Presiona 'q' para salir.")

    # Como el video no se detiene, usamos un ciclo 'while' para capturar todo
```

```
while True:

    # Aquí le pedimos a la cámara que nos dé la imagen que está viendo
    ahorita

    ok, cuadro = mi_video.read()

    if not ok:

        print("No se pudo leer la señal de la cámara.")

        break

    # Esta línea abre una ventanita para mostrarnos el video en la pantalla

    cv2.imshow('Mi Video en Vivo', cuadro)

    # Si el alumno (o usuario) presiona la tecla 'q', el video se detiene

    if cv2.waitKey(1) & 0xFF == ord('q'):

        break

    # Muy importante: al final hay que "soltar" la cámara para que se apague
    el foquito

    mi_video.release()

    cv2.destroyAllWindows()

    print("Cámara apagada correctamente.")

if __name__ == "__main__":

    usar_camara()
```

Conclusión.

Como conclusión, la programación de dispositivos de salida es fundamental ya que es el puente final entre la lógica y el mundo real. Se pudo determinar que, ya sea mediante una salida física como la de una impresora o una salida de flujo constante como la de una cámara de video, la clave reside en la traducción precisa de datos hacia el hardware. Comprender estos procesos es fundamental, ya que permite crear sistemas que interactúan de manera eficiente y directa con el entorno del usuario.

Referencias.

Euroinnova Business School. (2024). ¿Cómo funciona una cámara de vídeo?. Recuperado de <https://www.euroinnova.com/blog/como-functiona-una-camara-de-video>

IBM. (s.f.). Conceptos de impresora: Gestión de impresión y servidores. Recuperado de <https://www.ibm.com/docs/es/cmofm/9.5.0?topic=concepts-printer>

OpenCV. (2024). Getting Started with Video: Capture, Display, and Save. Recuperado de https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Oracle. (2024). Java Print Service User Guide. Recuperado de <https://docs.oracle.com/javase/8/docs/technotes/guides/jps/index.html>